NDSS 2026 论文配套代码 · 中文使用指南

轻量级 LLM 事故响应

本指南说明如何在 PyCharm 中运行、阅读与微调论文配套代码：微调、检索增强、前瞻规划三步方法与代码逐节对照，创新点速查，执行步骤与常见问题解答。

原论文：Hammar, Alpcan, Lupu · NDSS 2026 · DOI 10.14722/ndss.2026.240358

目 录

一、论文与代码概览 3

1.1 代码仓库结构 3

1.2 论文五项声明（Claims）与验证方式 4

二、方法三步走与代码创新点对照 4

2.1 第一步：指令微调（论文 §IV-B） 4

2.2 第二步：RAG 检索增强（论文 §IV-C，图 7） 5

2.3 第三步：前瞻规划与幻觉上界（论文 §IV-D、§V） 5

三、环境准备 6

四、用 PyCharm 打开项目 6

五、执行流程 S1-S4（对应论文附录 F） 7

5.1 S1：加载微调模型（验证 C2、C3） 7

5.2 S2：加载训练数据集（验证 C1） 7

5.3 S3：生成响应动作（验证 C3、C5） 7

5.4 S4：微调模型（演示训练流程，实验 E4） 8

六、实验结果速览（理解代码的意义） 8

七、常见问题 8

附录：论文关键内容速查表 9

一、论文与代码概览

本指南面向需要在本地运行、阅读与微调论文配套代码的读者。论文提出用14B 参数的轻量级大语言模型为网络安全事故响应提供决策支持：把系统日志翻译成可执行的响应计划，并通过指令微调、检索增强生成（RAG）、前瞻规划三步把"幻觉"（生成看似合理实则无效的响应动作）的概率控制在有理论保证的上界之内。相比基于提示工程的前沿大模型方案（OpenAI O3、Gemini 2.5 Pro），该方法更轻量、可本地部署、且恢复时间最多缩短 22%（论文 §VII）。

项目	内容
论文标题	Incident Response Planning Using a Lightweight Large Language Model with Reduced Hallucination
作者与单位	Kim Hammar、Tansu Alpcan（墨尔本大学）；Emil C. Lupu（帝国理工学院）
发表会议	NDSS Symposium 2026（网络与分布式系统安全研讨会），DOI: 10.14722/ndss.2026.240358
代码仓库	github.com/Kim-Hammar/llm_incident_response_ndss26（CC-BY-SA 4.0）
数据集 / 模型权重	huggingface.co/datasets/kimhammar/CSLE-IncidentResponse-V1（约 536MB）；huggingface.co/kimhammar/LLMIncidentResponse（约 15GB）
演示视频	youtube.com/watch?v=SCxq2ye-R4Y（无 GPU 时以此验证功能）

表 1 论文与工件信息（对应论文附录 F「工件附录」）

1.1 代码仓库结构

文件	作用	论文对应
load_fine_tuned_llm.py	步骤 S1：加载微调好的 14B 模型（基座 + LoRA 适配器约 201MB）	§IV-B；附录 F(S1)
load_training_dataset.py	步骤 S2：加载 68,000 条事故指令数据集	§IV-B；附录 F(S2)
response_generation.py	步骤 S3：随机抽取事故，流式生成响应动作	§IV-B 式(1)、附录 B/F(S3)
fine_tune_llm.py	步骤 S4：LoRA 微调（缩小版，5 条样本演示）	§IV-B 式(2)；Table 7；附录 F(S4)
preprint/paper.pdf	论文预印本（对照阅读用）	全文
llm_recovery（外部库）	RAG 管线与规划算法的实现库（pip 安装）	§IV-C/D；§V

表 2 仓库文件与论文章节对照

1.2 论文五项声明（Claims）与验证方式

声明	含义	验证途径
C1	发布首个事故响应微调数据集（68,000 条）	运行 S2
C2	发布首个开源的事故响应微调 LLM	运行 S1
C3	模型轻量，可在商用硬件运行	运行 S1/S3；论文 Table 4
C4	幻觉概率有界且可缩小（Prop. 1/2）	论文 §V 理论 + Fig. 14 实验
C5	生成的事故响应计划有效	运行 S3；论文 §VII 实验

表 3 论文附录 F「Major Claims」

二、方法三步走与代码创新点对照

论文方法分离线微调与在线规划两个阶段、三个步骤（论文 §IV-A，图 1/图 2）。下表给出"论文概念 – 代码位置 – 创新点"的三方对照，各脚本文件头部的注释中有更详细的分析。

步骤	论文出处	代码位置	创新点说明
1. 指令微调	§IV-B	fine_tune_llm.py；S1 加载权重	首个事故响应领域微调 LLM；测试床真实攻击日志 + 前沿模型合成数据混合造数；LoRA 让 14B 模型可在单卡微调
2. RAG 检索	§IV-C 图 7	llm_recovery 库	从日志自动提取失陷指标（IOC），调用 OTX 威胁情报 API 检索后拼入提示词，弥补模型训练数据滞后
3. 前瞻规划	§IV-D 算法 1	llm_recovery 库	N 个候选动作各模拟 M 条恢复轨迹，选期望恢复时间最短者（自我验证），并证明幻觉概率上界

表 4 三步方法总览（创新点浓缩表）

2.1 第一步：指令微调（论文 §IV-B）

训练数据 $D = \{(x^i, y^i)\}$ 共 68,000 对。指令 x 分两类：动作生成（生成下一步响应动作）与状态预测（判断遏制/评估/取证/驱逐/加固/恢复六阶段的完成度）；答案 y 附带链式思考（CoT）推理。数据来源：先在 64 台主机的 Docker 测试床上真实执行侦察、爆破、漏洞利用攻击（论文表 2、表 5、图 17），构造 500 对；再用 Gemini 2.5 Pro 与 OpenAI O3 合成扩充至 68,000 对，覆盖的 MITRE ATT&CK; 战术分布见论文图 4。

训练目标为答案上的逐 token 交叉熵损失（论文式 (2)），通过 LoRA 参数高效微调实现：只训练约 5033 万个低秩适配参数（运行时可见 "Trainable parameters: 50331648"），超参数与论文表 7 完全一致（rank 64、alpha 128、dropout 0.05、学习率 0.00095）。论文图 5 显示较高学习率

收敛到更低损失；图 6 显示微调把动作准确率从 0.16 提升到 0.89。

超参数	取值	说明
LoRA rank / alpha / dropout	64 / 128 / 0.05	低秩适配矩阵结构与正则化（论文表 7）
学习率	0.00095	论文图 5：高学习率收敛更好
温度 / 量化 / 轮数	0.6 / 4 bit / 2	论文表 7；缩小版脚本取 1 轮
可训练参数量	约 5033 万	相比 14B 全参数微调节省约 300 倍显存

表 5 微调超参数（与论文表 7 对照）

2.2 第二步：RAG 检索增强（论文 §IV-C，图 7）

微调数据止于历史事故，模型不认识新出现的漏洞。系统从日志自动提取失陷指标（如 CVE 编号、主机名），调用 AlienVault OTX 等威胁情报接口检索，把结果拼接进提示词后再让模型生成动作。论文给出的对比：针对 CVE-2021-44228（Log4Shell），无 RAG 时模型只会输出泛泛的"隔离主机"；有 RAG 时能给出针对该漏洞的具体缓解措施，从而缩短恢复时间。该管线在配套库 `llm_recovery` 中实现，本仓库的脚本负责演示其中模型侧的生成能力。

2.3 第三步：前瞻规划与幻觉上界（论文 §IV-D、§V）

响应过程被形式化为随机最短路径问题：恢复状态 s_t 是六维向量（遏制、评估、取证、驱逐、加固、恢复，各维取值 0 到 1，论文定义 1）；恢复时间 T 是到达终态 $(1,1,1,1,1,1)$ 的累计动作耗时（定义 2，式 (4)）。若某动作不改变期望恢复时间-to-go，即视为"幻觉动作"（定义 4）。

算法 1（论文第 8 页）在每个时间步：采样 N 个候选动作；对每个动作用 LLM 自模拟 M 条恢复轨迹，估计期望恢复时间 $\tilde{Q}(s,a)$；选择 $\tilde{Q}$ 最小的动作（式 (5)）。这种"自我验证"相当于让动作接受模型自身预测结果的一致性检验，从而过滤幻觉。

理论上（论文 §V）：只要候选中存在非幻觉动作，且非幻觉动作带来的时间收益 δ 大于模型预测误差带来的混淆 $2\cdot\eta\cdot||J||_\infty\cdot(||\tilde{J}||_\infty+1)$，则算法 1 必选非幻觉动作（命题 1）；而"全部候选都是幻觉"的概率可用 Hoeffding 不等式约束：$P(h \geq h+\varepsilon) \leq \exp(-2\cdot\varepsilon^2\cdot L)$，其中 L 为估计幻觉率的采样数（命题 2）。增大候选数 N 与采样数 M/L 即可把幻觉概率压到任意小，代价是规划时间线性增加（论文图 13 显示并行化后可近似常数）。

13.46 vs 16.21

平均恢复时间（离散时间单位，越低越好）：本方法 13.46，次优方法 16.21；相比前沿大模型最多缩短 22%（论文图 12，§VII-A）

三、环境准备

唯一的外部依赖是论文作者发布的 Python 库 llm_recovery（PyPI 可装），RAG 管线、LoRA 微调循环与规划算法都在该库中。

> 安装依赖
>
> ```
> pip install llm_recovery==0.0.13 # Python 3.10+
> pip install llm_recovery==0.0.7 # Python 3.9 或更早版本
> ```

步骤	硬件要求（论文表 8）	存储
S2 加载数据集	普通 CPU，8GB 内存	约 1GB
S1 加载微调模型	普通 CPU，72GB 内存	约 15GB
S3 生成响应动作	GPU（RTX 8000 级别），72GB 内存	约 16GB
S4 微调（缩小版）	单块 GPU 即可	约 16GB
S4 微调（完整复现）	4 块 RTX 8000，训练约 800 分钟（论文图 5）	约 200GB

表 6 硬件与存储需求（对应论文表 8）

> 存储有限的建议：本机只保留代码仓库（约 2.7MB）。S2 数据集约 536MB 按需下载；S1/S3/S4 涉及约 15GB 模型权重，磁盘紧张时可跳过，直接观看官方演示视频 youtube.com/watch?v=SCxq2ye-R4Y 来验证功能。数据集与权重托管在 Hugging Face 与 Zenodo，脚本首次运行时自动下载。

四、用 PyCharm 打开项目

以下步骤在 PyCharm（社区版即可）中完成，一次配置后四个脚本均可直接运行。

1. 打开项目 PyCharm 菜单 File - Open，选择 llm_incident_response_ndss26 文件夹，点击 Trust Project。

2. 配置解释器 File - Settings - Project - Python Interpreter，点击齿轮选择 Add Interpreter，建议新建 Virtualenv 环境（Python 3.8 及以上），确定后等待索引完成。

3. 安装依赖 打开底部 Terminal（View - Tool Windows - Terminal），执行第三章的 pip install 命令。

4. 设置编码 Settings - Editor - File Encodings，将 Global/Project Encoding 均设为 UTF-8，避免中文注释乱码（脚本均含中文注释）。

5. 运行脚本 在项目树中右键目标脚本（如 response_generation.py），选择 Run；或打开文件后点击 main 函数旁的绿色三角。四个脚本均无需命令行参数。

6. 观察下载 首次运行 S1 会下载约 15GB 权重、S2/S3/S4 需要约 536MB 数据集，进度显示在 Run 窗口。下载内容存放在用户目录的 Hugging Face 缓存中，不占用仓库空间。

7. 无 GPU 情形 S3/S4 会自动退回 CPU，仅能验证流程、速度极慢；建议直接看演示视频核对预期输出。

五、执行流程 S1-S4（对应论文附录 F）

四个脚本按顺序执行；每步的期望输出与论文第 20 页 Experiment E1-E4 的结果一致。

5.1 S1：加载微调模型（验证 C2、C3）

```
python load_fine_tuned_llm.py
# 期望输出：
# Loading the fine-tuned incident response LLM.
# LLM loaded successfully on device: cuda:0 （无 GPU 时为 cpu）
```

脚本从 Hugging Face 拉取 kimhammar/LLMIncidentResponse：14B 基座 DeepSeek-R1-Distill-Qwen-14B 外加大约 201MB 的 LoRA 适配器，半精度加载让内存占用减半。

5.2 S2：加载训练数据集（验证 C1）

```
python load_training_dataset.py
# 期望输出：
# Training dataset loaded successfully. Number of instructions: ...
# 随后打印第一条指令样例（论文附录 B 提示词模板的实际内容）
```

指令样例的结构：### System（系统描述）、### Logs（Snort/Wazuh 日志）、### Incident（事故摘要）、### State（六维恢复状态）、### Previous recovery actions（已执行动作）、### Instruction（任务要求）。该模板正是论文附录 B 展示的提示词格式。

5.3 S3：生成响应动作（验证 C3、C5）

```
python response_generation.py
# 期望输出（示例，与论文 E3 一致）：
# I recognize that while the attack is contained, I do not yet have
# enough information to fully eradicate it. Therefore, I choose to
# acquire disk and memory images along with relevant logs, preserving
# evidence in a forensically sound manner to support analysis.</think>
# {
# "Action": "Acquire full disk and memory images of 10.20.11.42 and
# export DNS, firewall, and NetFlow logs to write-protected
# storage.",
# "Explanation": "Capturing images and logs secures evidence for later
# analysis and legal requirements."
# }
```

输出分两段：think 标签内的链式思考，以及随后的 JSON 动作对象。Action 必须具体、可执行、不重复已执行动作，Explanation 说明选择理由——这些约束都来自微调数据中的指令模板（论文附录 B）。温度 0.8 的随机采样使每次运行结果不同，也对应论文算法 1 中"同一状态可采样出多个候选动作"的机制。

5.4 S4：微调模型（演示训练流程，实验 E4）

```
python fine_tune_llm.py
# 期望输出：
# Trainable parameters: 50331648
# Step: 1, Epoch: 0.5000, Progress: 50.0%, Avg_loss=..., LR=..., ...
# Step: 2, Epoch: 1.0000, ...
```

脚本默认只用前 5 条样本快速验证训练闭环；要去掉 instructions 与 answers 两行的 [0:5] 切片即切换为全量训练（完整复现需 4 块 RTX 8000 与数天时间）。超参数与论文表 7 一致。

六、实验结果速览（理解代码的意义）

以下结果帮助读者理解"这套代码为什么值得跑"。详见论文 §VII 与图 12 至图 16。

方法	平均恢复时间（越低越好）	参数规模
本方法（14B 微调 + 规划）	13.46	14B（论文表 4）
Gemini 2.5 Pro	16.21	未知（不少于 100B）
OpenAI O3	17.28	未知（不少于 100B）
DeepSeek-R1（未微调）	17.09	671B

表 7 与前沿大模型对比（论文图 12，25 起事故 + 5 起误报，5 个随机种子）

消融实验（论文图 15）：去掉微调后平均恢复时间从 13 升到 25，去掉规划升到 21，去掉 RAG 影响较小——微调与规划是性能的主要来源。

与强化学习对比（论文图 16）：本方法与 PPO 在两个仿真环境中表现相当，但 PPO 需要针对每个事故训练约 10–20 分钟，本方法无需按事件训练。

幻觉分析（论文图 14）：经验幻觉概率始终低于命题 2 的理论上界（置信度 0.99），且增大候选动作数 N 与采样数 M 可进一步压低。

七、常见问题

问：Hugging Face 下载失败或很慢？

答：设置镜像后重跑：Windows 下 set HF_ENDPOINT=https://hf-mirror.com；macOS/Linux 下 export HF_ENDPOINT=https://hf-mirror.com。

问：内存不足以加载 14B 模型？

答：14B 半精度约需 28-30GB 内存或显存。不够时可观看演示视频核对输出；或参考论文表 7 的 4-bit 量化思路，在加载代码中加入 bitsandbytes 量化参数。

问：想完整复现论文结果？

答：去掉 fine_tune_llm.py 中的 [0:5] 切片做全量微调；评估流程见论文附录 F 与 CSLE 仓库（github.com/Limmen/csle）。规划算法参数：候选动作 N=3、模拟轨迹 M=3（论文 §VII）。

问：如何引用本文？

答：Hammar K, Alpcan T, Lupu E C. Incident response planning using a lightweight large language model with reduced hallucination[C]//Proceedings of the NDSS Symposium 2026. San Diego, 2026. DOI: 10.14722/ndss.2026.240358。

问：许可与致谢？

答：原代码与数据遵循 CC-BY-SA 4.0（见仓库 LICENSE.md）。本仓库为学习用途的注释版，版权归原作者 Kim Hammar、Tansu Alpcan、Emil C. Lupu 所有。

附录：论文关键内容速查表

内容	论文位置	一句话说明
图 1 方法总览	第 1 页	微调、检索、规划三步流程图
图 2 两阶段架构	第 3 页	离线微调 + 在线日志分析与规划
定义 1 恢复状态	第 6 页 式(3)	六维进度向量（MITRE D3FEND）
定义 2 恢复时间	第 6 页 式(4)	到达终态的累计动作耗时
定义 4 幻觉动作	第 7 页	不减少期望恢复时间-to-go 的动作
算法 1 规划流程	第 8 页	N 候选 × M 轨迹，选 $\tilde{Q}$ 最小者
命题 1 充分条件	第 9 页	δ 大于预测误差影响时必选非幻觉动作
命题 2 概率上界	第 9 页	Hoeffding 上界 $\exp(-2 \cdot \varepsilon^2 \cdot L)$
表 7 超参数	第 18 页	LoRA 与 PPO 超参数
表 8 硬件要求	第 20 页	各工件的硬件门槛
图 12 主实验	第 11 页	恢复时间/无效动作/失败率对比
图 15 消融	第 11 页	三步各自的贡献

表 8 论文关键定义、算法与实验速查